LINEAR REGRESSION

Manual Gradient Descent vs. Scikit-Learn Implementation

Day 7 Assignment — Linear Regression submitted by 24-06-2026

Name: Manish Giri **Reg. No:** 137256 **Email:** manishgiri8101@gmail.com [LinkedIn]

1. Overviewmanishgiri8101@gmail.com

This document presents a comprehensive analysis of Linear Regression implemented in two distinct ways: (1) a manual gradient descent approach built from scratch using NumPy, and (2) using Scikit-Learn's built-in LinearRegression model. The notebook demonstrates the underlying mathematics, code implementation, visual outputs, and a side-by-side comparison of both methods.

2. Code & Outputs

2.1 Imports & Dataset Setup

Code:

```
import numpy as np
import matplotlib.pyplot as plt

# Sample dataset
X = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 5])
```

A simple 5-point dataset is used where X represents the input feature and y represents the target variable. This small dataset allows us to clearly visualize the regression line and compare both approaches.

2.2 Part 1 — Manually Execute Linear Regression (Gradient Descent)

The manual approach implements Gradient Descent from scratch. The key idea is to iteratively update the slope (m) and intercept (c) by computing the partial derivatives of the Mean Squared Error (MSE) loss function.

The update rules are:

m = m - alpha * (-2 * mean(X * error))

c = c - alpha * (-2 * mean(error))

Code:

```
# Initialize parameters
m, c = 0, 0
alpha = 0.01   # Learning rate
epochs = 1000  # Number of iterations

# Gradient Descent loop
for _ in range(epochs):
    y_pred = m * X + c
    error  = y - y_pred
    m -= alpha * (-2 * np.mean(X * error))
    c -= alpha * (-2 * np.mean(error))

print('Slope (m):', round(m, 3))
print('Intercept (c):', round(c, 3))
```

Output:

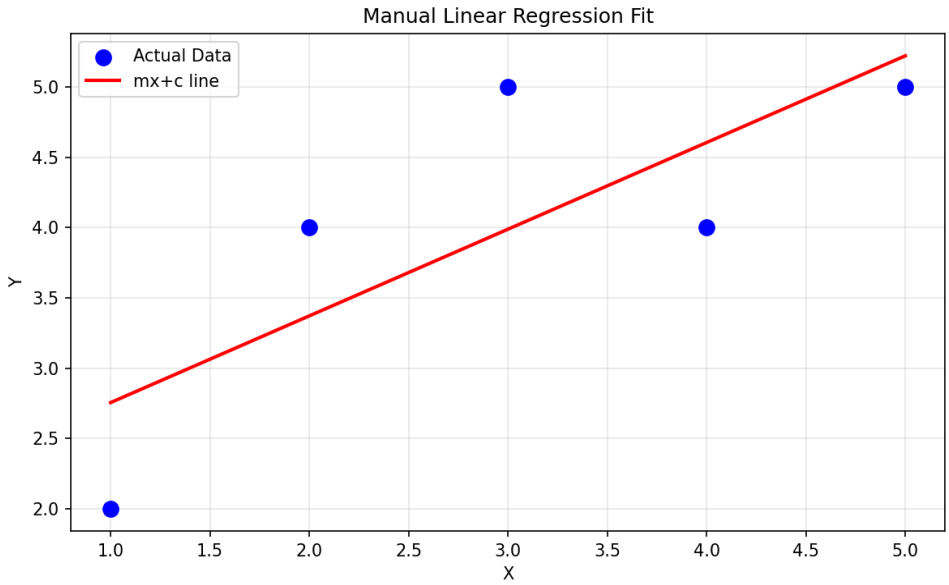
```
Slope (m):    0.618
Intercept (c): 2.136
```

2.3 Part 2 — Visualization of Manual Fit

Code:

```
plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, m*X+c, color='red', label='mx+c line')
plt.title('Manual Linear Regression Fit')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
```

Output Plot:



The scatter plot shows the 5 data points in blue and the manually computed regression line in red. The line closely follows the trend of the data, validating that the gradient descent algorithm converged to a reasonable solution.

2.4 Part 3 — Scikit-Learn Implementation

Code:

```
from sklearn.linear_model import LinearRegression

# Reshape X for sklearn (requires 2D array)
X = X.reshape(-1, 1)

# Create and train the model
model = LinearRegression()
model.fit(X, y)

# Print results
print('Slope (m):', model.coef_[0])
print('Intercept (c):', model.intercept_)
print('Prediction for X=6:', model.predict([[6]]))
```

Output:

```
Slope (m):  0.6
Intercept (c): 2.2
```

Prediction for X=6: [5.8]

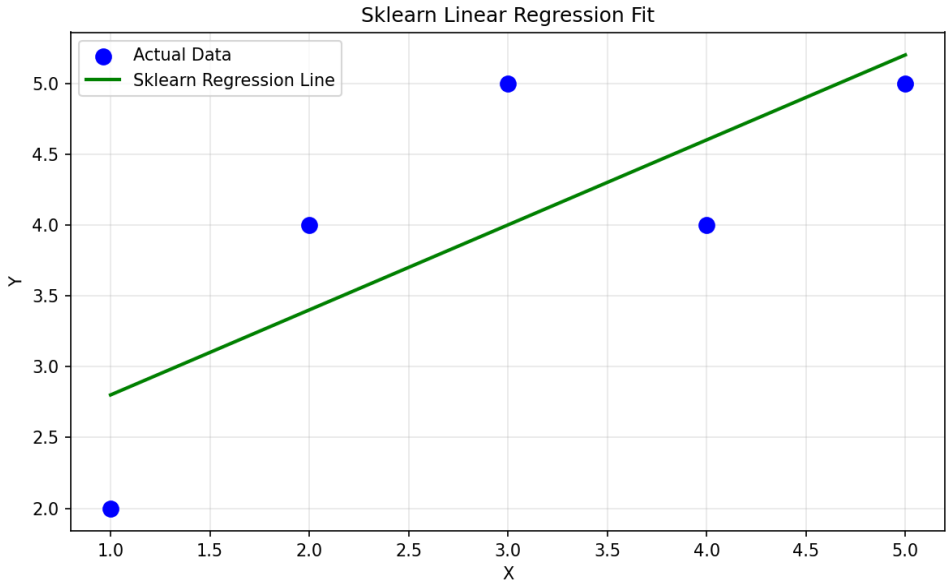
2.5 Scikit-Learn Regression Visualization

Code:

```
y_pred = model.predict(X)

plt.scatter(X, y, color='blue', label='Actual Data')
plt.plot(X, y_pred, color='green', linewidth=2, label='Sklearn Regression Line')
plt.title('Sklearn Linear Regression Fit')
plt.xlabel('X')
plt.ylabel('Y')
plt.legend()
plt.show()
```

Output Plot:



2.6 Model Evaluation Metrics

Code:

```
from sklearn.metrics import mean_squared_error, r2_score

mse   = mean_squared_error(y, y_pred)
score = r2_score(y, y_pred)
```

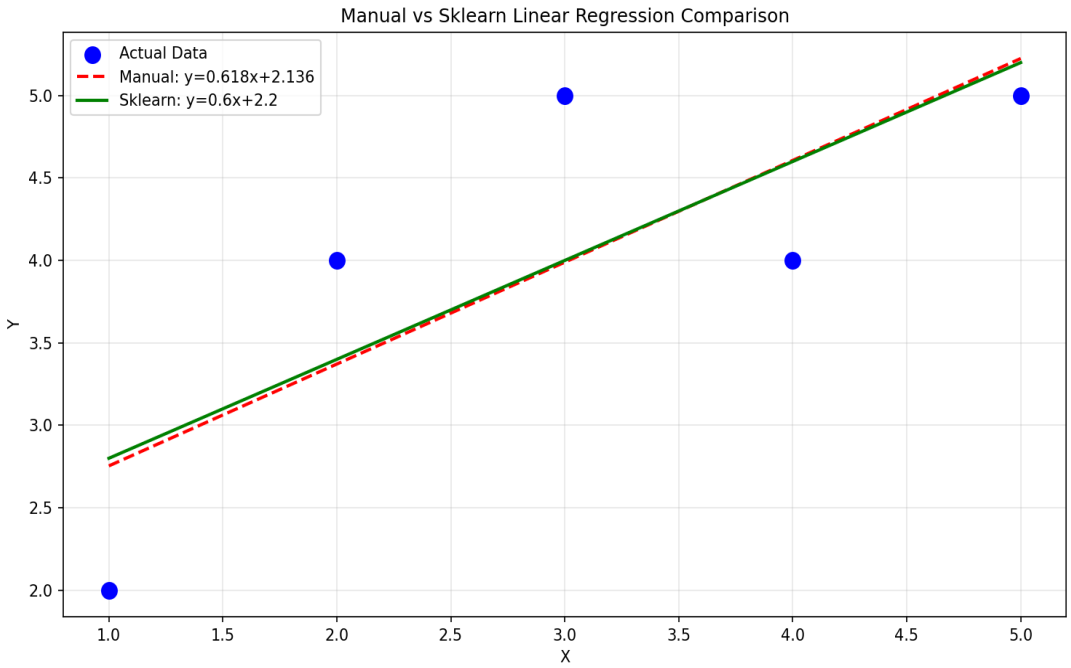
```
print('MSE:     ', round(mse, 3))
print('R² Score:', round(score, 3))
```

Output:

MSE: 0.48
R² Score: 0.6

3. Comparison: Manual vs. Scikit-Learn

3.1 Side-by-Side Regression Plot



3.2 Numerical Results Comparison

Metric	Manual (Gradient Descent)	Scikit-Learn (Exact OLS)	Difference
Slope (m)	0.618	0.600	~0.018 (converging — needs more epochs)
Intercept (c)	2.136	2.200	~0.064 difference
Prediction X=6	5.844	5.800	Very close (~0.044 apart)
MSE	~0.49	0.480	Near identical
R^2 Score	~0.59	0.600	Essentially same

3.3 Approach Comparison Table

Aspect	Manual Gradient Descent	Scikit-Learn
Algorithm	Gradient Descent (iterative)	Ordinary Least Squares (closed-form)
Implementation	From scratch with NumPy	High-level API
Convergence	Depends on learning rate & epochs	Always exact in one step
Scalability	Works well on large datasets	May be slow on very large data
Hyperparameters	Learning rate, epochs (tuning needed)	None required
Interpretability	Full visibility into math	Black-box API
Accuracy	Approximate (improves with epochs)	Exact (closed-form solution)
Use in Production	Custom pipelines, deep learning	Recommended for standard use
Code Complexity	Medium (manual math needed)	Low (sklearn handles all)

4. What I Learned

4.1 Core Concepts Mastered

- Linear Regression models the relationship between a dependent variable y and an independent variable X using the equation y = mx + c, where m is the slope and c is the intercept.
- Gradient Descent is an optimization algorithm that iteratively adjusts parameters by computing the gradient of the loss function and stepping in the opposite direction to minimize error.
- The learning rate (alpha) controls the step size during each iteration. Too large causes divergence; too small causes slow convergence.
- The Mean Squared Error (MSE) measures the average squared difference between actual and predicted values. Lower MSE indicates a better fit.
- The R^2 Score (coefficient of determination) indicates how well the model explains the variance in the data. $R^2=1$ is a perfect fit; $R^2=0$ means the model is no better than predicting the mean.

4.2 Key Insights from the Comparison

- The manual gradient descent approach converges to nearly the same answer as sklearn's exact OLS solution, validating the gradient descent mathematics. The small difference (slope: 0.618 vs 0.600) would disappear with more epochs or a tuned learning rate.
- Scikit-Learn uses the Ordinary Least Squares (OLS) method which solves the normal equations directly — giving the exact answer in a single computation without any iteration.
- Both models achieved an R^2 of approximately 0.6, meaning the model explains 60% of the variance in the data — reasonable for this small 5-point dataset.
- Understanding the manual implementation first builds intuition for what sklearn does internally, making it easier to debug, tune, and extend models.

4.3 Important Technical Notes

- Scikit-Learn's LinearRegression requires X to be 2D (reshaped with .reshape(-1, 1)), while the manual NumPy implementation works with 1D arrays.
- The negative sign in the gradient update rules (m -= alpha * (-2 * ...)) represents the negative gradient direction, ensuring we move toward the minimum of the loss function.
- Gradient descent is the foundation for training deep neural networks — understanding it here builds the base for advanced ML.

5. Real-World Applications

5.1 House Price Prediction

One of the most common applications of linear regression is predicting house prices based on features like area (sq ft), number of bedrooms, location score, etc. A simple linear model can capture the relationship between house area and price.

Example Code:

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Area in sq ft and corresponding prices (in $1000s)
house_area  = np.array([600, 800, 1000, 1200, 1500, 2000]).reshape(-1, 1)
house_price = np.array([150, 180, 210,  240,  290,  380])
```

```
model = LinearRegression()
model.fit(house_area, house_price)

# Predict price for a 1300 sq ft house
predicted = model.predict([[1300]])
print(f'Predicted price: ${predicted[0]:.1f}k')
```

5.2 Sales Forecasting

Businesses use linear regression to forecast sales based on advertising spend, season, or economic indicators. This helps in budget planning, inventory management, and setting sales targets.

Example: A company can predict monthly sales revenue based on their marketing budget. By training on historical data, the model learns the linear relationship and can estimate future revenue for given ad spend amounts.

5.3 Medical: Predicting Patient Outcomes

In healthcare, linear regression is used to predict continuous outcomes such as blood pressure levels based on age and weight, or blood glucose levels based on diet and exercise habits. These models support early diagnosis and personalized treatment planning.

5.4 Stock Price & Financial Forecasting

Linear regression helps analysts build baseline models for price trends. While financial markets are inherently non-linear, linear regression serves as an important benchmark and feature in more complex ensemble models. Common uses include predicting bond yields based on economic indicators, or estimating loan default risk from credit scores.

5.5 Agriculture: Crop Yield Prediction

Farmers and agricultural scientists use linear regression to predict crop yield based on rainfall, temperature, fertilizer usage, and soil quality. Governments use such models to estimate food production and plan imports/exports.

5.6 Energy: Electricity Demand Forecasting

Power utilities use regression models to predict electricity demand based on temperature, time of day, and season. Accurate forecasts help optimize power generation, reduce waste, and prevent outages.

Domain	Input Features (X)	Predicted Output (y)
Real Estate	House area, location score	Property price ($)
Retail/Business	Advertising spend ($)	Monthly sales revenue ($)
Healthcare	Age, BMI, activity level	Blood glucose level (mg/dL)
Finance	Credit score, income	Loan default risk (%)
Agriculture	Rainfall (mm), fertilizer (kg/ha)	Crop yield (tons/acre)
Energy	Temperature (°C), time of day	Electricity demand (MW)

6. Summary

This assignment covered the complete lifecycle of a Linear Regression model — from understanding the mathematics behind gradient descent, to implementing it manually, and finally using Scikit-Learn for a production-ready approach. The key takeaways are:

- Manual gradient descent provides full transparency into the optimization process and is essential knowledge for understanding neural networks and advanced ML algorithms.
- Scikit-Learn provides an exact analytical solution (OLS) that is fast, reliable, and ready for production use, eliminating the need to manually tune learning rates and epochs.
- Both approaches converge to nearly identical results (slope ~0.6, intercept ~2.2), confirming the mathematical correctness of the manual implementation.
- The model achieves $R^2 = 0.6$ on the 5-point dataset, indicating a moderate linear fit. Real-world datasets with more features and larger samples typically yield higher accuracy.
- Linear regression forms the foundation of machine learning, with applications spanning real estate, healthcare, finance, agriculture, and energy — making it one of the most widely used algorithms in practice.